Near-Data Accelerator Extension for a VeeR EL2 RISC-V SoC

Abstract

Problem Statement

Conventional von Neumann systems keep computation and data storage separate, so data has to move through the memory hierarchy before the CPU can process it. Onur Mutlu's work on the data-movement bottleneck shows why this becomes important for data-intensive workloads: the cost of accessing and moving data can become much larger than the simple computation performed on it.

Reduction operations such as sum, min, max, and argmax are a simple example of this mismatch. Each element is read once and needs only a small amount of computation. This makes reduction a useful workload for studying whether memory-intensive work can be decoupled from the CPU and handled by a dedicated engine.

Proposed SoC Architecture

The proposed SoC is built around the provided VeeR EL2 RV32IMC RISC-V processor and the required SoC peripherals. A custom **Near-Data Accelerator (NDA)** is added as an independent AXI4 master with direct access to main memory. The processor sends a compact reduction command to the NDA rather than processing every element itself. Once started, the NDA reads the operand data from memory and performs the reduction autonomously, returning only the final result to the processor.

The design is intentionally kept small. We are not assuming that the NDA is always better than software. The workload size is part of the experiment: for small arrays, setup and interrupt overhead may make software better, while larger arrays may give the NDA enough work to justify the extra hardware. The goal is to find and measure this tradeoff rather than retrofit the hardware around a desired result.

Selected IPs

The SoC integrates three custom Intellectual Property (IP) blocks to facilitate high-efficiency data processing and monitoring:

IP Block	Functionality
NDA Reduction Engine	Supports sum, min, max, and argmax operations on arrays stored in main memory.
Hardware Performance Monitor (HPM)	Records CPU cycles, memory bus transactions, NDA activity, and software fallbacks.
NDA Watchdog	Monitors progress during operation; triggers software fallback if progress stalls beyond a timeout.

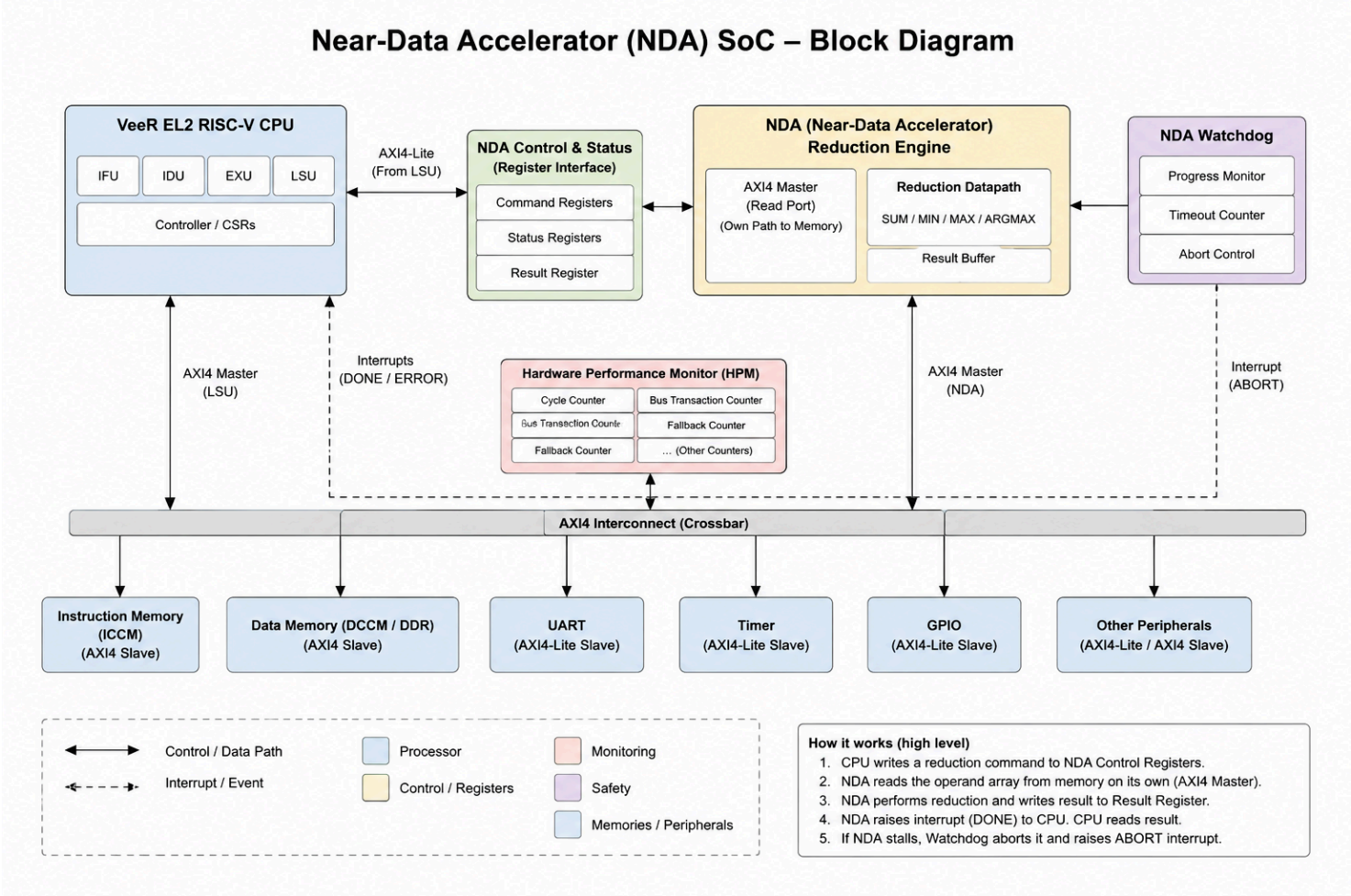
Software and Evaluation

A small software driver exposes the NDA operations to applications running on the RISC-V processor. After issuing a command, the processor does not need to poll the accelerator. The NDA runs independently and raises an interrupt when the operation completes or when the watchdog triggers a fallback, allowing the CPU to perform other work while the reduction is running.

The evaluation compares conventional software execution with NDA execution across different array sizes. We will measure execution time, CPU cycles, memory transactions, NDA activity, interrupt and setup overhead, and hardware resource cost. The aim is to determine where the NDA is useful, where software is better, and whether the performance benefit justifies the additional hardware.

Architecture Document

Block Diagram



The VeeR EL2 core is left unchanged. The NDA is added as another AXI4 master on the SoC interconnect. It has a control interface used by the CPU and a separate memory path used to read the reduction operands. The CPU sends the work description once, while the NDA handles the element-by-element processing on its own.

Driver and Execution Flow

The application chooses whether to use the software reduction or the NDA. For the NDA path:

1. **Initialization**: The driver writes the base address, length, operation type, and start command.
2. **Autonomous Execution**: The NDA processes the data independently of the CPU.
3. **Completion/Failure**: On completion, it raises an interrupt for the driver to read the result. If the watchdog reports a timeout, the driver falls back to the software implementation.

Interface Specifications

The NDA utilizes two primary interfaces:

- **Control/Status Interface**: AXI4-Lite slave used by the CPU to configure and start the operation and to read status and results.
- **Data Interface**: AXI4 master used by the NDA to read array data from main memory without CPU involvement for each element.

The control interface exposes registers for the starting address, number of elements, reduction operation, start/status, result, result index for argmax, and progress information for the watchdog.

Bus Protocol Details

- **NDA Data Path**: AXI4 master with burst reads from Data Memory. Once started, the NDA generates its own read addresses and processes the returned data.
- **NDA Control Path**: AXI4-Lite slave for small, infrequent control and status accesses from the CPU.
- **HPM and Watchdog**: AXI4-Lite slaves for configuration and counter/status access.
- **Completion**: NDA raises an interrupt when the operation completes. The CPU can therefore continue useful work while the NDA is running instead of continuously polling it.
- **Watchdog**: The watchdog detects lack of progress and prevents the NDA from starting another memory transaction after timeout. It does not interrupt an AXI burst that is already in progress.